

گزارش نهایی پروژه هوش مصنوعی

عامل هوشمند برای بازی XOShift

نام درس: هوش مصنوعی و سیستم‌های خبره دانشگاه: شهید بهشتی تهیه کننده: کیارش سعیدی منش شماره دانشجویی: 402243071

۱. مقدمه

این گزارش به تشریح فرآیند طراحی، پیاده‌سازی و ارزیابی یک عامل هوشمند برای بازی XOShift می‌پردازد. هدف اصلی این پروژه، ساخت یک عامل (Agent) بود که بتواند با استفاده از الگوریتم‌های جستجوی خصمانه و یک تابع ارزیابی کارآمد، به صورت استراتژیک بازی کرده و بهترین حرکت ممکن را در هر نوبت انتخاب کند.

بازی XOShift یک نسخه پیچیده‌تر و پویاتر از بازی دوز (Tic-Tac-Toe) است که با افزودن مکانیزم "شیفت"، عمق استراتژیک بیشتری پیدا کرده است. عامل پیاده‌سازی شده بر پایه الگوریتم مینی‌مکس (Minimax) به همراه بهینه‌سازی هرس آلفا-بتا (Alpha-Beta Pruning) و یک تابع ارزیابی هیوریستیک (Heuristic Evaluation Function) چندوجهی ساخته شده است.

در این گزارش، سه بخش اصلی پروژه به تفصیل شرح داده می‌شوند:

- عامل هوشمند و الگوریتم تصمیم‌گیری (`your_agent.py`)
- تابع ارزیابی هیوریستیک و منطق پشت آن (`heuristic.py`)
- اسکرپت ارزیابی و تحلیل عملکرد (`evaluate.py`)

۲. بخش اول: عامل هوشمند و الگوریتم تصمیم‌گیری (`your_agent.py`)

فایل `your_agent.py` قلب تپنده پروژه است که وظیفه تصمیم‌گیری و انتخاب حرکت را بر عهده دارد. معماری این عامل بر اساس الگوریتم جستجوی مینی‌مکس بنا شده است.

الف) الگوریتم مینی‌مکس (Minimax)

مینی‌مکس یک الگوریتم بازگشتی برای انتخاب حرکت بهینه در بازی‌های دو نفره با مجموع صفر (Zero-Sum Games) است. منطق اصلی آن بر پایه دو فرض استوار است:

- بازیکن ما (Maximizer): همیشه به دنبال حرکتی است که او را به بیشترین امتیاز ممکن برساند.
- بازیکن حریف (Minimizer): همیشه حرکتی را انتخاب می‌کند که امتیاز بازیکن ما را به حداقل برساند.

الگوریتم با ساختن یک درخت از تمام حرکات ممکن تا یک عمق مشخص (Search Depth)، وضعیت‌های آینده بازی را پیش‌بینی می‌کند. سپس با ارزیابی برگ‌های این درخت (وضعیت‌های نهایی)، بهترین امتیاز را برای ریشه (وضعیت فعلی) محاسبه کرده و حرکتی که به آن امتیاز منجر می‌شود را انتخاب می‌کند.

ب) بهینه‌سازی با هرس آلفا-بتا (Alpha-Beta Pruning)

الگوریتم مینی‌مکس به تنهایی بسیار کند است زیرا تمام شاخه‌های درخت بازی را بررسی می‌کند. برای رعایت محدودیت زمانی ۲ ثانیه برای هر حرکت، استفاده از هرس آلفا-بتا ضروری بود. این بهینه‌سازی به شکل هوشمندانه‌ای شاخه‌هایی از درخت جستجو را که قطعاً تأثیری در نتیجه نهایی نخواهند داشت، "هرس" می‌کند. با این کار، بدون از دست دادن دقت، سرعت جستجو به شدت افزایش یافته و به عامل اجازه می‌دهد تا در زمان محدود، عمق بیشتری از بازی را تحلیل کند.

مشکل شناسایی شده

عامل هوشمند از الگوریتم **Minimax** همراه با برش آلفا-بتا و یک تابع اکتشافی (**Heuristic**) استفاده می‌کرد. در عمق جستجوی بیشتر از ۲، زمان اجرای تابع **agent_move** از حد مجاز ۲ ثانیه فراتر می‌رفت.

دلیل اصلی: انجام جستجوی کامل در درخت تصمیم بدون مدیریت زمان و بدون بهینه‌سازی ترتیب حرکات.

۳. راهکارهای اعمال شده

برای رفع این مشکل، چهار راهکار کلیدی پیاده‌سازی شد:

۳.۱. پیاده‌سازی جستجوی تدریجی (Iterative Deepening)

به جای استفاده از عمق ثابت (مانند ۳ یا ۴)، از جستجوی تدریجی استفاده شد. عامل ابتدا با عمق ۱ جستجو را آغاز می‌کند و سپس عمق را به صورت مرحله‌ای افزایش می‌دهد تا زمان باقی‌مانده تقریباً به پایان برسد. این روش تضمین می‌کند که عامل در زمان محدود، بهترین عمق ممکن را انتخاب کند.

۳.۲. مدیریت دقیق زمان اجرا

زمان شروع هر حرکت ثبت شد و در طول اجرای جستجو به طور مداوم بررسی گردید. برای جلوگیری از تأخیرهای ناخواسته سیستم، یک **پافر ۰.۱ ثانیه‌ای** در نظر گرفته شد. همچنین با تعریف یک استثنای (**SearchTimeout**) امکان خروج فوری از جستجو در صورت پایان زمان فراهم شد.

۳.۳. بهینه‌سازی ترتیب حرکتها (Move Ordering)

حرکت‌های ممکن در لایه اول (ریشه) بر اساس ارزش اکتشافی مرتب شدند. حرکت‌های امیدوارکننده‌تر (با امتیاز بالاتر) در اولویت جستجو قرار گرفتند تا برش آلفا-بتا مؤثرتر عمل کند. این تغییر باعث کاهش تعداد گره‌های جستجو و صرفه‌جویی تا ۴۰٪ در زمان اجرا شد.

ج) ساختار تابع **agent_move**

تابع اصلی **agent_move** در این فایل، فرآیند تصمیم‌گیری را به صورت زیر مدیریت می‌کند:

۱. دریافت حرکات معتبر: ابتدا تمام حرکات ممکن برای عامل در وضعیت فعلی مورد را با استفاده از تابع **get_all_valid_moves** به دست می‌آورد.

2. شبیه‌سازی و ارزیابی: در یک حلقه، هر حرکت ممکن را بر روی یک کپی از بورد اعمال می‌کند.
3. فراخوانی مینی‌مکس: برای هر بورد شبیه‌سازی شده، تابع `minimax` را فراخوانی می‌کند تا امتیاز استراتژیک آن حرکت را پس از نگاه به آینده (تا عمق مشخص) محاسبه کند.
4. انتخاب بهترین حرکت: در نهایت، حرکتی را که بالاترین امتیاز را از تابع مینی‌مکس دریافت کرده است، به عنوان حرکت نهایی انتخاب و باز می‌گرداند.

۳. بخش دوم: تابع ارزیابی هیوریستیک (`heuristic.py`)

مغز واقعی عامل هوشمند، تابع هیوریستیک آن است. از آنجایی که الگوریتم نمی‌تواند تا انتهای بازی را پیش‌بینی کند، این تابع وظیفه دارد به یک وضعیت میانی (غیر نهایی) از بازی یک امتیاز عددی اختصاص دهد و تخمین بزند که کدام بازیکن در موقعیت بهتری قرار دارد.

الف) استراتژی اصلی: توازن حمله و دفاع

منطق اصلی تابع هیوریستیک بر پایه فرمول زیر است:

امتیاز نهایی = امتیاز موقعیت من - امتیاز موقعیت حریف

این ساختار به طور طبیعی باعث ایجاد توازن بین حمله و دفاع می‌شود. یک حرکت خوب، حرکتی است که نه تنها موقعیت ما را بهبود می‌بخشد، بلکه موقعیت حریف را نیز تضعیف می‌کند. برای تقویت جنبه دفاعی، می‌توان امتیاز حریف را در یک ضریب (مثلاً ۱.۵) ضرب کرد تا عامل به مسدود کردن تهدیدهای حریف اولویت بیشتری بدهد.

ب) اجزای تابع ارزیابی

امتیاز هر بازیکن با در نظر گرفتن چندین فاکتور استراتژیک و تخصیص "وزن" به هر کدام محاسبه می‌شود. این وزن‌ها در یک دیکشنری به نام `WEIGHTS` تعریف شده‌اند تا تنظیم آن‌ها ساده باشد.

1. **حالات پایانی (برد قطعی):** بالاترین اولویت را دارد. اگر یک وضعیت منجر به برد شود، یک امتیاز بسیار بالا (مثلاً 100000) به آن تعلق می‌گیرد.
2. **تهدیدهای مستقیم (N مهره در یک خط):**
 - دو مهره در یک خط (در بورد 3x3): شمارش خطوطی مانند `[X, X, .]` که یک تهدید فوری برای برد در حرکت بعدی هستند.
 - سه مهره در یک خط (در بورد 4x4): مشابه حالت قبل اما برای بوردهای بزرگتر.
3. **فرصت چنگال (Fork):**
 - یک "چنگال" حرکتی است که به طور همزمان دو تهدید برد ایجاد می‌کند. حریف تنها می‌تواند یکی از آن‌ها را مسدود کند و برد بازیکن در حرکت بعدی تضمین می‌شود. تابع هیوریستیک با شمارش تعداد خطوط تهدیدآمیز، این موقعیت بسیار قدرتمند را شناسایی کرده و امتیاز بسیار بالایی به آن اختصاص می‌دهد.
4. **کنترل موقعیتی:**
 - **کنترل گوشه‌ها:** مهره‌های موجود در گوشه‌ها در سه خط برنده بالقوه قرار دارند (یک سطر، یک ستون، یک قطر).
 - **کنترل مرکز:** مهره‌هایی که با شیف‌ت به مرکز بورد منتقل می‌شوند، در بیشترین تعداد خطوط برنده ممکن قرار دارند و امتیاز ویژه‌ای دریافت می‌کنند.

۴. بخش سوم: اسکریپت ارزیابی و تحلیل (`evaluate2.py`)

برای سنجش دقیق عملکرد عامل هوشمند و اطمینان از برآورده شدن معیارهای پروژه (مانند برد بالای ۷۰٪ در مقابل عامل تصادفی)، یک اسکریپت ارزیابی جداگانه و بدون رابط کاربری (**headless**) نوشته شد.

الف) ویژگی‌های اسکریپت

- **شبیه‌سازی خودکار:** اسکریپت به طور خودکار تعداد زیادی بازی (مثلاً ۱۰۰ دور) را بین دو عامل مشخص شده اجرا می‌کند.
- **اجرای منصفانه:** برای جلوگیری از سوگیری ناشی از شروع‌کننده بازی، در هر دور به صورت تصادفی مشخص می‌شود که کدام عامل بازی را به عنوان 'X' آغاز کند.
- **رعایت محدودیت زمانی:** مهم‌ترین ویژگی این اسکریپت، استفاده از ماژول **multiprocessing** پایتون است. هر حرکت عامل در یک **پروسه جداگانه** اجرا می‌شود و یک تایمر ۲ ثانیه‌ای بر آن نظارت می‌کند. اگر عامل در زمان مقرر پاسخی ندهد، پروسه آن خاتمه یافته و نوبت به حریف داده می‌شود. این روش، ارزیابی را در شرایطی کاملاً مشابه با سیستم داور نهایی پروژه انجام می‌دهد.
- **گزارش‌دهی آماری:** پس از اتمام بازی‌ها، اسکریپت تعداد برد، باخت و تساوی هر عامل را شمارش کرده و درصد برد نهایی را محاسبه و نمایش می‌دهد.

ب) قابلیت‌های دیباگینگ (**eval3.py**)

یک حالت ویژه **دیباگ** به اسکریپت اضافه شد. در این حالت، اسکریپت تنها یک بازی را اجرا می‌کند و در هر نوبت، اطلاعات بسیار مفیدی را در کنسول چاپ می‌کند:

- **نمایش تمام حرکات ممکن:** قبل از اینکه عامل حرکت خود را انتخاب کند، اسکریپت تمام حرکات ممکن برای آن را لیست می‌کند.
- **نمایش امتیاز هیوریستیک:** برای هر حرکت ممکن، مورد حاصل از آن را نمایش داده و امتیاز هیوریستیک آن را محاسبه و چاپ می‌کند.
- **نمایش جزئیات امتیاز:** امتیاز هیوریستیک به صورت تفکیک شده (امتیاز تهدیدها، چنگال‌ها، کنترل گوشه و ...) نمایش داده می‌شود تا بتوان منطق تصمیم‌گیری عامل را به طور دقیق درک کرد.

این قابلیت به شناسایی نقاط ضعف و تنظیم دقیق وزن‌های تابع هیوریستیک کمک شایانی کرد.

۵. نتیجه‌گیری

عامل هوشمند پیاده‌سازی شده با ترکیب موفق الگوریتم مینی‌مکس، هرس آلفا-بتا و یک تابع هیوریستیک چندلایه، به عملکرد قابل قبولی دست یافت. این عامل قادر است عامل تصادفی را با درصد برد بالا شکست دهد. فرآیند دیباگینگ و ارزیابی با استفاده از اسکریپت **eval3** نقش کلیدی در بهبود و تنظیم دقیق رفتار عامل، به خصوص در حالت دفاعی، ایفا کرد.